Infrastructure for the Maintenance Queue

Vânia P. Vidal[1]

Universidade Federal do Ceará - UFC

1 Relational Schema of Maintenance Queue

The computed maintenance information (Rule Contributions) MUST be materialized in a predefined maintenance queue infrastructure composed of two relational tables:

- `rdf_maintenance_queue`. Stores one maintenance event per statement-level relational update;
- `rdf_rule_contribution`. Stores one rule contribution per maintenance event and per relevant transformation. rule ψ. Each row in `rdf_rule_contribution` represents the maintenance information computed for a single transformation rule.

The queue preserves the commit order of maintenance events and serves as the communication interface between:

- PostgreSQL AFTER triggers; and
- the asynchronous RDF maintenance worker.

1.1 Schema of `rdf_maintenance_queue`

```
CREATE TABLE rdf_maintenance_queue (
    event_id BIGSERIAL PRIMARY KEY,
    relation_name TEXT NOT NULL,
    operation_type TEXT NOT NULL,
    deleted_tuples JSONB NOT NULL,
    inserted_tuples JSONB NOT NULL,
    status TEXT NOT NULL DEFAULT 'pending',
    created_at TIMESTAMP NOT NULL DEFAULT now(),
    processed_at TIMESTAMP,
    error_message TEXT
);
```

1.2 Schema of `rdf_rule_contribution`

```
CREATE TABLE rdf_rule_contribution (
    event_id BIGINT NOT NULL REFERENCES rdf_maintenance_queue(event_id),
    rule_id TEXT NOT NULL,
    rule_graph_uri TEXT NOT NULL,
    relevance_type TEXT NOT NULL,
    pivot_relation TEXT NOT NULL,
    path JSONB NOT NULL,
    rule_contribution JSONB NOT NULL,
   PRIMARY KEY (event_id, rule_id)
);
```

Each row in `rdf_rule_contribution` represents the maintenance information computed for a single transformation rule: $\psi \in Relev(R)$, including:

- the affected tuples for deletion: $A^{-}[\psi](u)$;
- the affected tuples for insertion: $A^{+}[\psi](u)$;
- the post-update RDF contributions: $S2[\psi](u)$;
- the insertion changeset: $\Delta^{+}[\psi](u)$.

The rule contribution MUST preserve the semantics and provenance of the corresponding transformation rule ψ, including its associated named graph.

2 JSON Template to Store Rule Contributions

2.1 CTR Rule Contribution Template

A CTR (Class Transformation Rule) generates RDF class membership assertions. Formally, a CTR produces RDF quads of the form:

$$(s, rdf{:}type, C, g)$$

where:

- s = URI of the RDF resource generated from the pivot tuple;
- $rdf{:}type$ = RDF class-membership predicate;
- C = RDF class associated with the transformation rule;
- g = named graph URI corresponding to hasURI('http://example.com/resource/tr/<ψ>',ψ, g).

Since all RDF quads generated by a CTR share the same:

- predicate;
- RDF class;
- named graph,

these fixed components MUST be stored once in a dedicated template structure called:

class_quad_template

The generated RDF contributions store only the varying component:

- the RDF subject URI

The JSON schema for a CTR rule contribution is therefore:

```
{
  "rule_id": "...",
  "rule_type": "CTR",
  "pivot_relation": "...",
  "class_quad_template": {
    "predicate": "rdf:type",
    "class": "<RDF class URI>",
    "graph": "<hasURI('http://example.com/resource/tr/<$\psi$>',$\psi$, $g$)>"
  },
  "rdf_contributions": {
    "S2":[
      {
        "subject": "<resource URI>"
      }
    ],
    "DeltaPlus": [
      {
        "subject": "<resource URI>"
      }
    ]
  }
}
```

The asynchronous worker reconstructs the RDF quads generated by the CTR using:

- the subject values stored in: *rdf_contributions*
- the fixed RDF components stored in: *class_quad_template*

2.2 OTR Rule Contribution Template

An OTR (Object Transformation Rule) generates RDF object-property assertions between two RDF resources. Formally, an OTR produces RDF quads of the form:

$$(s, p, o, g)$$

where:

- s = URI of the RDF subject resource generated from the pivot tuple;
- p = RDF object-property predicate associated with the transformation rule;
- o = URI of the RDF object resource generated from a related tuple;
- g = named graph URI corresponding to hasURI('http://example.com/resource/tr/<ψ>',ψ, g)..

Since all RDF quads generated by an OTR share the same:

- predicate,
- named graph,

these fixed components MUST be stored once in a dedicated template structure called:

object_quad_template

The generated RDF contributions store only the varying components:

- the RDF subject URI;
- the RDF object URI.

The JSON schema for an OTR rule contribution is therefore:

```
{
  "rule_id": "...",
  "rule_type": "OTR",
  "affected_tuples": {
    "A_minus": [...],
    "A_plus": [...]
  },
  "object_quad_template": {
    "predicate": "<object-property URI>",
    "graph": "<hasURI('http://example.com/resource/tr/<$\psi$>',$\psi$,$g$)>"
  },
  "rdf_contributions": {
    "DeltaPlusPivot": [
      {
        "subject": "<subject resource URI>",
        "object": "<object resource URI>"
      }
    ],
    "S2": [
      {
        "subject": "<subject resource URI>",
        "object": "<object resource URI>"
      }
    ],
    "DeltaPlusRel": [
      {
        "subject": "<subject resource URI>",
        "object": "<object resource URI>"
      }
    ],
```

```
    "DeltaPlus": [
      {
        "subject": "<subject resource URI>",
        "object": "<object resource URI>"
      }
    ]
  }
}
```

The asynchronous worker reconstructs the RDF quads generated by the OTR using:

- the subject/object URI pairs stored in: *rdf_contributions*
- the fixed RDF components stored in: *object_quad_template*

For each RDF contribution stored in S2 or DeltaPlus, the worker reconstructs:

$$(subject, predicate, object, graph)$$

2.3 Example

If:

- subject = http://musicbrainz.org/track/1001
- predicate = http://purl.org/ontology/mo/performer
- object = http://musicbrainz.org/artist/555
- graph = https://ekg.example.org/mappings/psi_track_artist_01,

Then: the reconstructed RDF quad is:

```
GRAPH <https://ekg.example.org/mappings/psi_track_artist_01> {
  <http://musicbrainz.org/track/1001>
  <http://purl.org/ontology/mo/performer>
  <http://musicbrainz.org/artist/555> .
}
```

2.4 DTR Rule Contribution Schema

A DTR (Datatype Transformation Rule) generates RDF datatype-property assertions between an RDF resource and an RDF literal value. Formally, a DTR produces RDF quads of the form:

$$(s, p, o, g)$$

where:

- s = URI of the RDF subject resource generated from the pivot tuple;
- p = RDF datatype-property predicate associated with the transformation rule;
- o = RDF literal value generated from relational attribute values;
- g = named graph URI corresponding to hasURI('http://example.com/resource/tr/<ψ>',ψ, g).

Since all RDF quads generated by a DTR share the same:

- predicate;
- literal datatype;
- named graph,

these fixed RDF components MUST be stored once in a dedicated template structure called:

datatype_quad_template

The generated RDF contributions store only the varying components:

– the RDF subject URI;
– the RDF literal value.

The JSON schema for a DTR rule contribution is therefore:

```
{
        "rule_id": "...",
        "rule_type": "DTR",
        "affected_tuples": {
                "A_minus": [...],
                "A_plus": [...]
        },
        "datatype_quad_template": {
                "predicate": "<datatype-property URI>",
                "datatype": "<RDF datatype URI>",
                "graph": "<hasURI('http://example.com/resource/tr/<$\psi$>',$\psi$, $g$).>"
        },
        "rdf_contributions": {
                "DeltaPlusPivot": [
                {
                        "subject": "<subject resource URI>",
                        "object": "<literal value>"
                }
                ],
                "S2": [
                {
                        "subject": "<subject resource URI>",
                        "object": "<literal value>"
                }
                ],
                "DeltaPlusRel": [
                {
                        "subject": "<subject resource URI>",
                        "object": "<literal value>"
                }
                ],
                "DeltaPlus": [
                {
                        "subject": "<subject resource URI>",
                        "object": "<literal value>"
                }
                ]
        }
}
```

The asynchronous worker reconstructs the RDF quads generated by the DTR using:

– the subject/literal pairs stored in: *rdf_contributions*
– the fixed RDF components stored in: *datatype_quad_template*

For each RDF contribution stored in S2 or DeltaPlus, the worker reconstructs:

$$(subject, predicate, literal, graph)$$

2.5 Example:

If:

– subject = http://musicbrainz.org/track/1001

- predicate = http://purl.org/dc/terms/title
- object = "New Song"
- datatype = http://www.w3.org/2001/XMLSchema#string
- graph = https://ekg.example.org/mappings/psi_track_02

Then: the reconstructed RDF quad is:

```
GRAPH <https://ekg.example.org/mappings/psi_track_02> {
  <http://musicbrainz.org/track/1001>
  <http://purl.org/dc/terms/title>
  "New Song"^^xsd:string .
}
```